```
import pandas as pd
```

```
data = pd.read_csv('data/training.csv')
```

```
data.isna().sum().sum()
```

```
np.int64(0)
```

```
metrics = pd.DataFrame(columns=["model","Ca", "P", "pH", "SOC", "Sand","MCRMSE"])
```

U datasetu nema null vrednosti.

```
data.select_dtypes(include='str')
```

| | PIDN | Depth |
|---|---|---|
| **0** | XNhoFZW5 | Topsoil |
| **1** | 9XNspFTd | Subsoil |
| **2** | WDId41qG | Topsoil |
| **3** | JrrJf1mN | Subsoil |
| **4** | ZoIitegA | Topsoil |
| **...** | ... | ... |
| **1152** | bdcNNrbi | Topsoil |
| **1153** | 6HBVKZwh | Subsoil |
| **1154** | 5eLY5nw7 | Topsoil |
| **1155** | gsSGXhX6 | Subsoil |
| **1156** | A6IpBbfZ | Topsoil |

1157 rows × 2 columns

Izbacujemo PDIN koji predstavlja identifikator uzorka

```
data.drop('PIDN', inplace=True, axis=1)
```

Pretvaramo Depth obelezje u numericke vrednosti

```
data['Depth'] = data['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1})
```

# Provera korelacija

```
import numpy as np

cor_mat = data.corr()
cor_mat_ut = cor_mat.where(np.triu(np.ones(cor_mat.shape), k=1).astype(np.bool))
cols = cor_mat_ut.columns
correlated_columns = []
for column in cols:
    if any(cor_mat_ut[column] > 0.95):
        correlated_columns.append(column)
correlated_columns
```

```
data_bez_korelacija =  data.drop(correlated_columns, axis=1)
```

```
data_bez_korelacija
```

| | m7497.96 | BSAN | BSAV | CTI | ELEV | EVI | LSTD | LSTN | REF2 | |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0.302553 | -0.630435 | -0.783875 | -0.364146 | 1.165479 | 1.062682 | -0.716713 | -0.090016 | -0.537106 | -0. |
| **1** | 0.270192 | -0.630435 | -0.783875 | -0.364146 | 1.165479 | 1.062682 | -0.716713 | -0.090016 | -0.537106 | -0. |
| **2** | 0.317433 | -0.753623 | -0.929451 | -0.633972 | 1.544098 | 1.156705 | -1.282552 | -0.088336 | -0.631725 | -0. |
| **3** | 0.261116 | -0.753623 | -0.929451 | -0.633972 | 1.544098 | 1.156705 | -1.282552 | -0.088336 | -0.631725 | -0. |
| **4** | 0.260038 | -0.688406 | -0.884658 | -0.583576 | 1.276837 | 1.191691 | -1.206971 | 0.011420 | -0.528757 | -0. |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| **1152** | 0.207530 | -0.724638 | -0.772676 | 0.038610 | 1.501782 | 0.306851 | -0.511141 | -0.481023 | -0.940631 | -0. |
| **1153** | 0.182376 | -0.724638 | -0.772676 | 0.038610 | 1.501782 | 0.306851 | -0.511141 | -0.481023 | -0.940631 | -0. |
| **1154** | 0.146829 | -0.695652 | -0.795073 | -0.639966 | 1.287973 | 0.659621 | -0.549317 | -0.646528 | -0.515770 | -0. |
| **1155** | 0.091910 | -0.695652 | -0.795073 | -0.639966 | 1.287973 | 0.659621 | -0.549317 | -0.646528 | -0.515770 | -0. |
| **1156** | 0.471388 | -0.797101 | -0.783875 | 0.087648 | 1.265702 | 0.271137 | -0.245935 | -0.444938 | -0.914657 | -0. |

1157 rows × 19 columns

racunanje korelacije mi je ubilo operativni sistem

# Outlier-i

```
from scipy.stats import normaltest

norm = []
columns = data.select_dtypes(include='number').columns.tolist()
for column in columns:
    p = normaltest(data[column])
    if p.pvalue < 0.05:
        norm.append(False)
    else:
        norm.append(True)

print(pd.DataFrame(norm).sum(), data.shape)
```

```
0    418
dtype: int64 (1157, 3599)
```

Od 3599 kolona svega 418 ima normalnu raspodelu, pa ne mozemo koristiti z_score za uklanjanje outlier-a

```
data_numeric = data.select_dtypes(include='number')

data_bez_outliera = data

q1 = data_numeric.quantile(0.25)
q3 = data_numeric.quantile(0.75)
iqr = q3 - q1

columns = data_numeric.columns
for column in columns:
    if (column == 'Ca') | (column == 'P') | (column == 'pH') | (column == 'SOC') |
(column == 'Sand'):
        continue
    data_bez_outliera = data_bez_outliera[(data_bez_outliera[column] >= q1[column] - 1.5
* iqr[column]) & (data_bez_outliera[column] <= q3[column] + 1.5 * iqr[column])]
```

```
data_bez_outliera.shape
```

```
(715, 3599)
```

Izbacivanjem outliera izgubili smo preveliku kolicinu podataka, pa cemo ih clippovati

```
data_numeric = data.select_dtypes(include='number')

data_clipped = data

q1 = data_numeric.quantile(0.25)
q3 = data_numeric.quantile(0.75)
iqr = q3 - q1

columns = data_numeric.columns
for column in columns:
    if (column == 'Ca') | (column == 'P') | (column == 'pH') | (column == 'SOC') |
(column == 'Sand'):
        continue
    min = q1[column] - 1.5 * iqr[column]
    max = q3[column] + 1.5 * iqr[column]
    data_clipped[column] = data_clipped[column].clip(upper=max, lower=min)
```

# Modeli

Linearna regresija

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

model_lin_reg = LinearRegression(n_jobs=-1)
pipeline_lin_reg = Pipeline([
    ('scaler', StandardScaler()),
    ('model_lin_reg', model_lin_reg),
])
grid_params = {
    'scaler' : [None, StandardScaler()],
    'model_lin_reg__fit_intercept' : [False, True],
}

grid_lin_reg = GridSearchCV(pipeline_lin_reg, grid_params, n_jobs=-1, cv=5, scoring=
'neg_mean_squared_error')
```

Klasifikaciono stablo

```
from sklearn.tree import DecisionTreeRegressor

model_tree = DecisionTreeRegressor(random_state=7)
pipeline_tree = Pipeline([
    #('scaler', StandardScaler()),
    ('model_tree', model_tree),
])

parameters_tree = {
    "model_tree__ccp_alpha" : [0.0, 0.05, 0.0025],
    "model_tree__max_depth" : [2, 5, 7, 9, 11],
    "model_tree__criterion" : ['squared_error', 'friedman_mse', 'absolute_error'],
}

grid_tree = GridSearchCV(pipeline_tree, parameters_tree, n_jobs=-1, cv=5, scoring=
'neg_mean_squared_error')
```

KNN

```
from sklearn.neighbors import KNeighborsRegressor

knn = KNeighborsRegressor()

knn_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ("knn", knn),
])

parameters_knn = {
    "knn__n_neighbors" : [3, 5, 7, 9],
    "knn__weights" : ['uniform', 'distance'],
    "knn__metric" : ['minkowski'],
    "knn__algorithm" : [ 'kd_tree', 'brute']
}

grid_knn = GridSearchCV(knn_pipeline, parameters_knn, n_jobs=-1 , cv=5, scoring=
"neg_mean_squared_error")
```

Random Forest

```
from sklearn.ensemble import RandomForestRegressor

forest = RandomForestRegressor( n_estimators=100)

random_forest_pipeline = Pipeline([
    ("random_forest", forest),
])

parameters_random_forest = {
    "random_forest__n_estimators" : [50, 100, 150, 200],
    "random_forest__criterion" : ['squared_error', 'absolute_error', 'friedman_mse'],
    "random_forest__max_depth" : [2, 5, 7, 9]
}
grid_random_forest = GridSearchCV(random_forest_pipeline, parameters_random_forest,
n_jobs=-1 , cv=5, scoring="neg_mean_squared_error")
```

```
import xgboost as xgb

xg_boost_pipeline = Pipeline ([
    ("model_xg_boost", xgb.XGBRegressor(eval_metric='rmse',device='cuda'))
])

parameters_xg_boost = {
              "model_xg_boost__max_depth":    [4, 5, 6],
              "model_xg_boost__n_estimators": [500, 600, 700],
              "model_xg_boost__learning_rate": [0.01, 0.015]}

grid_xg_boost = GridSearchCV(xg_boost_pipeline, parameters_xg_boost, n_jobs=-1 , cv=5,
scoring="neg_mean_squared_error")
```

## Sa outlierima

Izdvajamo izlazna obelezja

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error

def the_funkcija(grid, trening_set, validacija_set, output_name):
    # izdvajanje izlaznih obelezja
    y_Ca = trening_set['Ca']
    y_P  = trening_set['P']
    y_pH = trening_set['pH']
    y_SOC = trening_set['SOC']
    y_Sand = trening_set['Sand']
    X = trening_set.drop(['Ca', 'P', 'pH', 'SOC', 'Sand'], axis=1)
    # train test split
    X_train_Ca, X_test_Ca, y_train_Ca, y_test_Ca = train_test_split(X, y_Ca, test_size=0.2
, random_state=7)
```

```
    X_train_P, X_test_P, y_train_P, y_test_P = train_test_split(X, y_P, test_size=0.2,
random_state=7)
    X_train_pH, X_test_pH, y_train_pH, y_test_pH = train_test_split(X, y_pH, test_size=0.2
, random_state=7)
    X_train_SOC, X_test_SOC, y_train_SOC, y_test_SOC = train_test_split(X, y_SOC,
test_size=0.2, random_state=7)
    X_train_Sand, X_test_Sand, y_train_Sand, y_test_Sand = train_test_split(X, y_Sand,
test_size=0.2, random_state=7)
    # krosvalidacija
    grid_Ca = grid.fit(X_train_Ca, y_train_Ca)
    best_model_Ca = grid_Ca.best_estimator_
    best_params_Ca = grid_Ca.best_params_

    grid_P = grid.fit(X_train_P, y_train_P)
    best_model_P = grid_P.best_estimator_
    best_params_P = grid_P.best_params_

    grid_pH = grid.fit(X_train_pH, y_train_pH)
    best_model_pH = grid_pH.best_estimator_
    best_params_pH = grid_pH.best_params_

    grid_SOC = grid.fit(X_train_SOC, y_train_SOC)
    best_model_SOC = grid_SOC.best_estimator_
    best_params_SOC = grid_SOC.best_params_

    grid_Sand = grid.fit(X_train_Sand, y_train_Sand)
    best_model_Sand = grid_Sand.best_estimator_
    best_params_Sand = grid_Sand.best_params_

    # kalkulacija rmse
    y_RMSE_Ca = root_mean_squared_error(best_model_Ca.predict(X_test_Ca), y_test_Ca)
    y_RMSE_P = root_mean_squared_error(best_model_P.predict(X_test_P), y_test_P)
    y_RMSE_pH = root_mean_squared_error(best_model_pH.predict(X_test_pH), y_test_pH)
    y_RMSE_SOC = root_mean_squared_error(best_model_SOC.predict(X_test_SOC), y_test_SOC)
    y_RMSE_Sand = root_mean_squared_error(best_model_Sand.predict(X_test_Sand),
y_test_Sand)

    # validacija
    prediction = pd.DataFrame()
    prediction['PIDN'] = validacija_set['PIDN']
    validacija_set.drop('PIDN', inplace=True, axis=1)

    prediction['Ca'] = pd.DataFrame(best_model_Ca.set_params(**best_params_Ca).fit(X,
y_Ca).predict(validacija_set))
    prediction['P'] = pd.DataFrame(best_model_P.set_params(**best_params_P).fit(X, y_P).
predict(validacija_set))
    prediction['pH'] = pd.DataFrame(best_model_pH.set_params(**best_params_pH).fit(X,
y_pH).predict(validacija_set))
    prediction['SOC'] = pd.DataFrame(best_model_SOC.set_params(**best_params_SOC).fit(X,
y_SOC).predict(validacija_set))
    prediction['Sand'] = pd.DataFrame(best_model_Sand.set_params(**best_params_Sand).fit
(X, y_Sand).predict(validacija_set))
```

```
    prediction.to_csv("predictions/" + output_name + ".csv", index=False)

    return {
        "model" : output_name,
        "Ca" : y_RMSE_Ca,
        "P" : y_RMSE_P,
        "pH": y_RMSE_pH,
        "SOC": y_RMSE_SOC,
        "Sand": y_RMSE_Sand,
        "MCRMSE" : (y_RMSE_Ca + y_RMSE_P + y_RMSE_pH + y_RMSE_SOC + y_RMSE_Sand)/5,
    }
```

Linearna regresija inicijalizacija

```
sorted_test = pd.read_csv('data/sorted_test.csv')
test_lin_reg = sorted_test.copy()
test_lin_reg['Depth'] = test_lin_reg['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1})
```

```
row = the_funkcija(grid_lin_reg, data, test_lin_reg, "lin_reg")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'lin_reg', 'Ca': 0.5049645752975458, 'P': 1.6576700943521183, 'pH':
0.7543211407602292, 'SOC': 0.7051158437573091, 'Sand': 0.7294245874734764, 'MCRMSE':
0.8702992483281358}
```

Regresiono stablo inicijalizacija

```
test_tree = sorted_test.copy()
test_tree['Depth'] = test_tree['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1})
```

```
row = the_funkcija(grid_tree, data, test_tree, "tree")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'tree', 'Ca': 0.5648514501553454, 'P': 1.0588995128583363, 'pH':
0.6745977967977185, 'SOC': 0.5302018230051582, 'Sand': 0.5241819155477588, 'MCRMSE':
0.6705464996728635}
```

KNN inicijalizacija

```
test_knn = sorted_test.copy()
test_knn['Depth'] = test_knn['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1})
```

```
row = the_funkcija(grid_knn, data, test_knn, "knn")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'knn', 'Ca': 0.3707050255092884, 'P': 0.7548587025994591, 'pH':
0.5371621638791397, 'SOC': 0.4639591210947606, 'Sand': 0.4528866776324533, 'MCRMSE':
0.5159143381430202}
```

## Bez outliera

```
test_lin_reg_c = sorted_test.copy()
test_lin_reg_c['Depth'] = test_lin_reg_c['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1})
```

```
row = the_funkcija(grid_lin_reg, data_clipped, test_lin_reg_c, "lin_reg_clipped")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'lin_reg_clipped', 'Ca': 0.5049645752975458, 'P': 1.6576700943521183, 'pH':
0.7543211407602292, 'SOC': 0.7051158437573091, 'Sand': 0.7294245874734764, 'MCRMSE':
0.8702992483281358}
```

Zakljucujemo da linearna regresija trenirana na clippovanim podacima radi isto

```
test_tree_clipped = sorted_test.copy()
test_tree_clipped['Depth'] = test_tree_clipped['Depth'].replace({'Topsoil' : 0 , 'Subsoil'
 : 1})
```

```
row = the_funkcija(grid_tree, data_clipped, test_tree_clipped, "tree_clipped")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'tree_clipped', 'Ca': 0.5648514501553454, 'P': 1.0588995128583363, 'pH':
0.6745977967977185, 'SOC': 0.5302018230051582, 'Sand': 0.5241819155477588, 'MCRMSE':
0.6705464996728635}
```

```
test_knn_clipped = sorted_test.copy()
test_knn_clipped['Depth'] = test_knn_clipped['Depth'].replace({'Topsoil' : 0 , 'Subsoil'
: 1})
```

```
row = the_funkcija(grid_knn, data_clipped, test_knn_clipped, "knn_clipped")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'knn_clipped', 'Ca': 0.3707050255092884, 'P': 0.7548587025994591, 'pH':
0.5371621638791397, 'SOC': 0.4639591210947606, 'Sand': 0.4528866776324533, 'MCRMSE':
0.5159143381430202}
```

# Bez korelisanih kolona

```
test_lin_reg_bk = sorted_test.copy()
test_lin_reg_bk['Depth'] = test_lin_reg_bk['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1
})
test_lin_reg_bk = test_lin_reg_bk.drop(correlated_columns, axis=1)
```

```
row = the_funkcija(grid_lin_reg, data_bez_korelacija, test_lin_reg_bk,
"lin_reg_bez_korelacija")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'lin_reg_bez_korelacija', 'Ca': 0.8329978812820192, 'P': 0.6988197190977719,
'pH': 0.6801143967048152, 'SOC': 0.8315303328421029, 'Sand': 0.7703700255047135,
'MCRMSE': 0.7627664710862845}
```

```
test_tree_bk = sorted_test.copy()
test_tree_bk['Depth'] = test_tree_bk['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1})
test_tree_bk = test_tree_bk.drop(correlated_columns, axis=1)
```

```
row = the_funkcija(grid_tree, data_bez_korelacija, test_tree_bk, "tree_bez_korelacija")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'tree_bez_korelacija', 'Ca': 0.5108288448240199, 'P': 0.7250483854005842, 'pH':
0.6163802595768648, 'SOC': 0.722614972588117, 'Sand': 0.5259174484953628, 'MCRMSE':
0.6201579821769898}
```

```
test_knn_bk = sorted_test.copy()
test_knn_bk['Depth'] = test_knn_bk['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1})
test_knn_bk = test_knn_bk.drop(correlated_columns, axis=1)
```

```
row = the_funkcija(grid_knn, data_bez_korelacija, test_knn_bk, "knn_bez_korelacija")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'knn_bez_korelacija', 'Ca': 0.6794565588462201, 'P': 0.7635625755021704, 'pH':
0.56808776830409, 'SOC': 0.6291466105575492, 'Sand': 0.543697402020544, 'MCRMSE':
0.6367901830461147}
```

```
test_rf_bk = sorted_test.copy()
test_rf_bk['Depth'] = test_rf_bk['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1})
test_rf_bk = test_rf_bk.drop(correlated_columns, axis=1)
```

```
row = the_funkcija(grid_random_forest, data_bez_korelacija, test_rf_bk,
"random_forest_bez_korelacija")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'random_forest_bez_korelacija', 'Ca': 0.5183382005402082, 'P':
0.8734598316520532, 'pH': 0.5159026567492003, 'SOC': 0.5596661231310657, 'Sand':
0.41526883901966255, 'MCRMSE': 0.576527130218438}
```

```
test_xg_boost_bk = sorted_test.copy()
test_xg_boost_bk['Depth'] = test_xg_boost_bk['Depth'].replace({'Topsoil' : 0 , 'Subsoil'
: 1})
test_xg_boost_bk = test_xg_boost_bk.drop(correlated_columns, axis=1)
test_xg_boost_bk['Depth'] = pd.to_numeric(test_xg_boost_bk['Depth'], errors='raise')
```

```
# data_xg_boost_bk = data.copy()
#
# data_xg_boost_bk['Depth'] = pd.to_numeric(data_xg_boost_bk['Depth'], errors='raise')
# data_xg_boost_bk.info()
# print(the_funkcija(grid_xg_boost, data_xg_boost_bk, test_xg_boost_bk,
"xg_boost_bez_korelacija"))
```

## Polynomial features

broj polinoma je prevelik pre izbacivanja korelisanih kolona, pa samo ovde koristimo polynomial features

ostavili smo samo drugi stepen polinoma jer visi stepeni daju isti rezulatat

```
from sklearn.preprocessing import PolynomialFeatures

pipeline_lin_reg_polynomial = Pipeline([
    ('scaler', StandardScaler()),
    ('feature_generator', PolynomialFeatures()),
    ('model_lin_reg', model_lin_reg),
])
lin_reg_params_polynomial = {
    'scaler' : [None, StandardScaler()],
    'feature_generator' : [None, PolynomialFeatures(degree=2)],
    'model_lin_reg__fit_intercept' : [False, True],
}

grid_lin_reg_polynomial = GridSearchCV(pipeline_lin_reg_polynomial,
lin_reg_params_polynomial, n_jobs=-1, cv=5, scoring='neg_mean_squared_error')
```

```
test_lin_reg_bk_pol = sorted_test.copy()
test_lin_reg_bk_pol['Depth'] = test_lin_reg_bk_pol['Depth'].replace({'Topsoil' : 0 ,
'Subsoil' : 1})
test_lin_reg_bk_pol = test_lin_reg_bk_pol.drop(correlated_columns, axis=1)
```

```
row = the_funkcija(grid_lin_reg_polynomial, data_bez_korelacija, test_lin_reg_bk_pol,
"lin_reg_bez_korelacija_polynomial")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'lin_reg_bez_korelacija_polynomial', 'Ca': 0.6420833025861913, 'P':
0.6988197190977719, 'pH': 0.6189733116872884, 'SOC': 0.766177253697109, 'Sand':
0.6706059297434439, 'MCRMSE': 0.6793319033623609}
```

```
pipeline_tree_polynomial = Pipeline([
    #('scaler', StandardScaler()),
    ('feature_generator', 'passthrough'),
    ('model_tree', model_tree),
])

parameters_tree_polynomial = {
    "model_tree__ccp_alpha" : [0.0, 0.05, 0.0025],
    "model_tree__max_depth" : [2, 5, 7, 9, 11],
    "model_tree__criterion" : ['squared_error', 'friedman_mse', 'absolute_error'],
    "feature_generator" : ['passthrough', PolynomialFeatures(degree=2)]
}

grid_tree_polynomial = GridSearchCV(pipeline_tree_polynomial, parameters_tree_polynomial,
n_jobs=-1, cv=5, scoring='neg_mean_squared_error')
```

```
test_tree_bk_pol = sorted_test.copy()
test_tree_bk_pol['Depth'] = test_tree_bk_pol['Depth'].replace({'Topsoil' : 0 , 'Subsoil'
: 1})
test_tree_bk_pol = test_tree_bk_pol.drop(correlated_columns, axis=1)
```

```
row = the_funkcija(grid_tree_polynomial, data_bez_korelacija, test_tree_bk_pol,
"tree_bez_korelacija_polynomial")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'tree_bez_korelacija_polynomial', 'Ca': 0.5108288448240199, 'P':
0.7250483854005842, 'pH': 0.6163802595768648, 'SOC': 0.722614972588117, 'Sand':
0.5259174484953628, 'MCRMSE': 0.6201579821769898}
```

```
knn_pipeline_polynomial = Pipeline([
    ('scaler', StandardScaler()),
    ('feature_generator', PolynomialFeatures()),
    ("knn", knn),
])

parameters_knn_polynomial = {
    "knn__n_neighbors" : [3, 5, 7, 9],
    "knn__weights" : ['uniform', 'distance'],
    "knn__metric" : ['minkowski'],
    "knn__algorithm" : [ 'kd_tree', 'brute'],
    "feature_generator" : [None, PolynomialFeatures(degree=2)]
}

grid_knn_polynomial = GridSearchCV(knn_pipeline_polynomial, parameters_knn_polynomial,
n_jobs=-1 , cv=5, scoring="neg_mean_squared_error")
```

```
test_knn_bk_pol = sorted_test.copy()
test_knn_bk_pol['Depth'] = test_knn_bk_pol['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1
})
test_knn_bk_pol = test_knn_bk_pol.drop(correlated_columns, axis=1)
```

```
row = the_funkcija(grid_knn_polynomial, data_bez_korelacija, test_knn_bk_pol,
"knn1_bez_korelacija_polynomial")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'knn1_bez_korelacija_polynomial', 'Ca': 0.6794565588462201, 'P':
0.7635625755021704, 'pH': 0.56808776830409, 'SOC': 0.6291466105575492, 'Sand':
0.543697402020544, 'MCRMSE': 0.6367901830461147}
```

```
random_forest_pipeline_polynomial = Pipeline([
    ('feature_generator', PolynomialFeatures()),
    ("random_forest", forest),
])

parameters_random_forest_polynomial = {
    "random_forest__n_estimators" : [50, 100, 150, 200],
    "random_forest__criterion" : ['squared_error', 'absolute_error', 'friedman_mse'],
    "random_forest__max_depth" : [2, 5, 7, 9],
    "feature_generator" : [None, PolynomialFeatures(degree=2)]
}
grid_random_forest_polynomial = GridSearchCV(random_forest_pipeline_polynomial,
parameters_random_forest_polynomial, n_jobs=-1 , cv=5, scoring="neg_mean_squared_error")
```

```
test_rf_bk_pol = sorted_test.copy()
test_rf_bk_pol['Depth'] = test_rf_bk_pol['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1})
test_rf_bk_pol = test_rf_bk_pol.drop(correlated_columns, axis=1)
```

```
row = the_funkcija(grid_random_forest_polynomial, data_bez_korelacija, test_rf_bk_pol,
"random_forest_bez_korelacija_polynomial")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'random_forest_bez_korelacija_polynomial', 'Ca': 0.5419239949229041, 'P':
0.9040560106789066, 'pH': 0.4970980974073057, 'SOC': 0.5581759282318434, 'Sand':
0.4252737368096484, 'MCRMSE': 0.5853055536101216}
```

Selekcija

```
from sklearn.feature_selection import SequentialFeatureSelector
selector = SequentialFeatureSelector(model_lin_reg, n_features_to_select=6, direction=
'forward')
pipeline_lin_reg_selection = Pipeline([
    ('scaler', StandardScaler()),
    ('feature_selector', selector),
    ('model_lin_reg', model_lin_reg),
])
lin_reg_params_selection = {
    'scaler' : [None, StandardScaler()],
    "feature_selector__direction" : ['forward', 'backward'],
    'model_lin_reg__fit_intercept' : [False, True],
}

grid_lin_reg_selection = GridSearchCV(pipeline_lin_reg_selection,
lin_reg_params_selection, n_jobs=-1, cv=5, scoring='neg_mean_squared_error')
```

```
test_lin_reg_bk_sel = sorted_test.copy()
test_lin_reg_bk_sel['Depth'] = test_lin_reg_bk_sel['Depth'].replace({'Topsoil' : 0 ,
'Subsoil' : 1})
test_lin_reg_bk_sel = test_lin_reg_bk_sel.drop(correlated_columns, axis=1)
```

```
row = the_funkcija(grid_lin_reg_selection, data_bez_korelacija, test_lin_reg_bk_sel,
"lin_reg_bez_korelacija_selection")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'lin_reg_bez_korelacija_selection', 'Ca': 0.8283647579786129, 'P':
0.6920284204098044, 'pH': 0.6822460263000224, 'SOC': 0.8581477133798922, 'Sand':
0.8011313451718595, 'MCRMSE': 0.7723836526480382}
```

```
selector = SequentialFeatureSelector(knn, n_features_to_select=6, direction='forward')
knn_pipeline_selection = Pipeline([
    ('scaler', StandardScaler()),
    ('feature_selector', selector),
    ("knn", knn),
])

parameters_knn_selection = {
    "knn__n_neighbors" : [3, 5, 7, 9],
    "knn__weights" : ['uniform', 'distance'],
    "knn__metric" : ['minkowski'],
    "knn__algorithm" : [ 'kd_tree', 'brute'],
    "feature_selector__direction" : ['forward', 'backward'],
}

grid_knn_selection = GridSearchCV(knn_pipeline_selection, parameters_knn_selection,
n_jobs=-1 , cv=5, scoring="neg_mean_squared_error")
```

```
test_knn_bk_sel = sorted_test.copy()
test_knn_bk_sel['Depth'] = test_knn_bk_sel['Depth'].replace({'Topsoil' : 0 , 'Subsoil' : 1
})
test_knn_bk_sel = test_knn_bk_sel.drop(correlated_columns, axis=1)
```

```
row = the_funkcija(grid_knn_selection, data_bez_korelacija, test_knn_bk_sel,
"knn_bez_korelacija_selection")
print(row)
metrics=pd.concat([metrics, pd.DataFrame([row])])
```

```
{'model': 'knn_bez_korelacija_selection', 'Ca': 0.5816067254419746, 'P':
0.6865540026830881, 'pH': 0.5371188972767886, 'SOC': 0.6045728658702846, 'Sand':
0.41334396574408644, 'MCRMSE': 0.5646392914032444}
```

```
metrics.to_csv("metrics.csv", index=False)
```